Lab 4 – Galaga Repair Lab: Complete Chat History

Repository: https://github.com/diya-122/SE-Lab-Reports

Project Folder: `Lab4_Galaga/`

Language: Python 3.13 / Pygame 2.6.1

Date: 15 October 2026

Table of Contents

- Task 1 – Fix the Cubic Bézier Bug
- Task 2 – Wave-Based Enemy Colors (`enemy_tint`)
- Task 3 – Wave-Start Banner (`on_wave_start`)
- Task 4 – Shield Charges (`shield_charges`)
- Final Commit History

Task 1 – Fix the Cubic Bézier Bug

User Prompt

> Inspect `game.py` and fix only the deliberate bug in the `bezier()` function.
>
> Compare its four weighted terms with the standard cubic Bézier formula:
>
> B(t) = (1-t)^3 P0 + 3(1-t)^2 t P1 + 3(1-t)t^2 P2 + t^3 P3
>
> Check the exponents of `u` and `t` in both middle terms. Correct any mistakes while preserving the rest of the code.
>
> Before editing, explain exactly what is wrong and why it causes incorrect enemy entry and dive paths. Make the smallest possible change. Do not implement any of the other three features yet.

Explanation of the Bug

The standard cubic Bézier formula requires the following four weighted terms:

Term	Correct exponents	Code (original)	Status
P0 term	`u³`	`u ** 3`	■ Correct
P1 term	`u² · t¹`	`u * u * t`	■ Correct
P2 term	`u¹ · t²`	`u * t`	■ **Wrong** – missing one `* t`
P3 term	`t³`	`t ** 3`	■ Correct

The third term was written as `3 * u * t * p2[...]` but the correct formula requires `3 * u * t² * p2[...]`. Because the `t²` factor was just `t`, the curve was incorrectly weighted at every point: at any given `t` the position was pulled too strongly toward `P2` and not smoothly toward `P3`. This distorted **both the arcing entry paths and the dive paths** — enemies flew to wrong positions on screen.

Code Before (lines 12–13)

```
x = u ** 3 * p0[0] + 3 * u * u * t * p1[0] + 3 * u * t * p2[0] + t ** 3 * p3[0]
y = u ** 3 * p0[1] + 3 * u * u * t * p1[1] + 3 * u * t * p2[1] + t ** 3 * p3[1]
```

Code After (lines 12–13)

```
x = u ** 3 * p0[0] + 3 * u * u * t * p1[0] + 3 * u * t * t * p2[0] + t ** 3 * p3[0]
y = u ** 3 * p0[1] + 3 * u * u * t * p1[1] + 3 * u * t * t * p2[1] + t ** 3 * p3[1]
```

Change: Added exactly one `* t` to the P2 term on both the `x` and `y` lines.

Diff

```
- x = u ** 3 * p0[0] + 3 * u * u * t * p1[0] + 3 * u * t * p2[0] + t ** 3 * p3[0]
- y = u ** 3 * p0[1] + 3 * u * u * t * p1[1] + 3 * u * t * p2[1] + t ** 3 * p3[1]
+ x = u ** 3 * p0[0] + 3 * u * u * t * p1[0] + 3 * u * t * t * p2[0] + t ** 3 * p3[0]
+ y = u ** 3 * p0[1] + 3 * u * u * t * p1[1] + 3 * u * t * t * p2[1] + t ** 3 * p3[1]
```

Testing

The game was launched with `python game.py`. It started successfully with no runtime errors. Pygame 2.6.1 initialized correctly.

```
pygame 2.6.1 (SDL 2.28.4, Python 3.13.6)
Hello from the pygame community. https://www.pygame.org/contribute.html
```

Git Push

Commit hash: `a67af73`

Branch: `main`

Files changed: 2 (`Lab4_Galaga/game.py`, `Lab4_Galaga/README.md`)

Insertions: 327

Commit message:

```
Lab4_Galaga: fix cubic Bezier P2 exponent bug in bezier()

The third weighted term in bezier() used u*t (i.e. u*t^1) instead of
the correct u*t^2 required by the standard cubic Bezier formula:

  B(t) = (1-t)^3*P0 + 3(1-t)^2*t*P1 + 3(1-t)*t^2*P2 + t^3*P3

Changed both x and y lines:
  Before: 3 * u * t * p2[...]
  After:  3 * u * t * t * p2[...]

This caused distorted enemy entry arcs and dive paths at runtime.
```

Push output:

```
To https://github.com/diya-122/SE-Lab-Reports.git
   b06e35d..a67af73  main -> main
```

Task 2 – Wave-Based Enemy Colors (`enemy_tint`)

User Prompt

> Now implement Task 2 only: `enemy_tint(kind)` in `game.py`.
>
> Requirements:
>
> 1. Support enemy kinds "boss", "red", and "blue".
> 2. Return an RGB tuple to override the default enemy color, or None to use `ENEMY_COLORS[kind]`.
> 3. Make enemy colors change based on the current wave number, so later waves look visually distinct.
> 4. Inspect the current game code to determine how the wave number is stored and how `enemy_tint()` is called.
> 5. Preserve the existing game mechanics and the Bézier fix from Task 1.
> 6. Do not implement `on_wave_start()` or `shield_charges()` yet.

Implementation Plan

Key observations from inspecting the code:

- `enemy_tint(kind)` takes only `kind` (line 17) — **no wave parameter**.
- Call site is line 192: `color = enemy_tint(enemy.kind) or ENEMY_COLORS[enemy.kind]` — returning `None` (falsy) falls back to `ENEMY_COLORS[kind]`.

• Wave number is stored as `self.wave` on the `Game` instance — not accessible from a module-level function.

• **Solution:** Add a module-level `_current_wave` variable updated inside `spawn_wave(wave)` — the single natural touch-point called on every wave transition.

Code Changes

1. Module-level variable (added after `ENTRY_TIME`):

```
# Tracks the current wave number so that enemy_tint() can read it
# without changing the function's signature or the call site in draw().
_current_wave = 1
```

2. `enemy_tint()` implementation (replaced `pass` stub):

```
def enemy_tint(kind):
    """Return an (r, g, b) colour override for an enemy kind, or None for the default.

    Colors shift every 2 waves to give each wave a visually distinct palette:
      tier 0 (wave 1-2)  : default hues  – green boss, red, blue
      tier 1 (wave 3-4)  : warm shift    – yellow-green boss, orange-red, cyan
      tier 2 (wave 5-6)  : hot shift     – gold boss, deep crimson, violet
      tier 3 (wave 7+)   : extreme shift – white-hot boss, magenta, ice-white
    """
    tier = min((_current_wave - 1) // 2, 3)

    TINTS = {
        "boss": [
            (90,  220,  90),   # tier 0 – lime green
            (180, 220,  40),   # tier 1 – yellow-green
            (220, 180,   0),   # tier 2 – gold
            (255, 255, 160),   # tier 3 – white-hot
        ],
        "red": [
            None,              # tier 0 – use default ENEMY_COLORS
            (240, 130,  30),   # tier 1 – orange
            (200,  20,  60),   # tier 2 – deep crimson
            (255,   0, 180),   # tier 3 – magenta
        ],
        "blue": [
            None,              # tier 0 – use default ENEMY_COLORS
            ( 40, 220, 220),   # tier 1 – cyan
            (160,  60, 240),   # tier 2 – violet
            (200, 230, 255),   # tier 3 – ice-white
        ],
    }
    return TINTS.get(kind, [None] * 4)[tier]
```

3. `spawn_wave()` update (set `_current_wave` at top):

```
def spawn_wave(wave):
    global _current_wave
    _current_wave = wave      # ← added
    enemies, index = [], 0
    ...
```

Color Palette by Wave Tier

Tier	Waves	Boss	Red enemies	Blue enemies
0	1–2	Lime green `(90,220,90)`	Default `(230,70,70)`	Default `(80,140,240)`
1	3–4	Yellow-green `(180,220,40)`	Orange `(240,130,30)`	Cyan `(40,220,220)`
2	5–6	Gold `(220,180,0)`	Deep crimson `(200,20,60)`	Violet `(160,60,240)`
3	7+	White-hot `(255,255,160)`	Magenta `(255,0,180)`	Ice-white `(200,230,255)`

> **Note:** Returning `None` for red/blue at tier 0 lets the `or ENEMY_COLORS[kind]` fallback on line 227 handle waves 1–2 naturally.

> The boss low-HP purple override `(170, 90, 220)` is applied **after** `enemy_tint()` and continues to work correctly.

Verification Results

All 6 checks passed before pushing:

Check	Result
Valid RGB / None for all kinds at all tiers	■ Pass
`_current_wave` updates correctly in `spawn_wave()`	■ Pass
Waves 1–2 preserve default red/blue colors	■ Pass
Later tiers change colors correctly	■ Pass
Boss low-HP override still works	■ Pass
Bézier fix preserved	■ Pass

Testing

Game launched with no errors:

```
pygame 2.6.1 (SDL 2.28.4, Python 3.13.6)
Hello from the pygame community. https://www.pygame.org/contribute.html
```

Git Push

Commit hash: `f5eaeb3`

Branch: `main`

Files changed: 1 (`Lab4_Galaga/game.py`)

Insertions: 37 | **Deletions:** 2

Commit message:

```
Lab4_Galaga: Task 2 – implement wave-aware enemy_tint()

Added wave-based color tinting for all enemy kinds without changing
the enemy_tint(kind) signature or its call site in draw().

Changes:
1. Added module-level _current_wave = 1 variable so that
   enemy_tint() can read the current wave without a parameter change.

2. Implemented enemy_tint(kind) with a 4-tier palette that shifts
   every 2 waves:
     tier 0 (wave 1-2): lime green boss / default red / default blue
     tier 1 (wave 3-4): yellow-green boss / orange red / cyan blue
     tier 2 (wave 5-6): gold boss / deep crimson red / violet blue
     tier 3 (wave 7+) : white-hot boss / magenta red / ice-white blue
   Returning None for red/blue at tier 0 preserves the original
   ENEMY_COLORS fallback on line 227.

3. Added `global _current_wave; _current_wave = wave` at the top of
   spawn_wave() – the single natural update point called on every
   wave transition (reset and wave clear).

Boss low-HP purple override and Bezier fix (Task 1) are preserved.
```

Push output:

```
To https://github.com/diya-122/SE-Lab-Reports.git
   a67af73..f5eaeb3  main -> main
```

Task 3 – Wave-Start Banner (`on_wave_start`)

User Prompt

Implement Task 3 only in `game.py`: `on_wave_start(wave)`.

>

Requirements:
1. Display a visible "WAVE N" banner when a new wave starts.
2. Keep the banner visible for approximately 1 second.
3. Use the existing Pygame drawing and timing architecture.
4. Do not use `time.sleep()` or any blocking delay.
5. Ensure the banner appears on the initial wave and every subsequent wave.
6. Reset the banner timer correctly for each new wave.
7. Do not implement Task 4 (`shield_charges`) yet.
8. Do not modify unrelated gameplay mechanics.

Implementation Plan

Key observations from inspecting the code:

- `on_wave_start(wave)` is called at line 119 inside `spawn_wave()` — after the enemy list is built. It takes only `wave`, no `self` reference.
- The game loop ticks `dt`-based timing — no blocking delays anywhere.
- `draw()` already renders conditional overlays (e.g., GAME OVER label at line 244).
- **Solution:** Same module-level bridge pattern as Task 2. A `_banner` dict holds `{"text": "", "timer": 0.0}`. `on_wave_start()` sets it, `update()` ticks it down, `draw()` renders it when `timer > 0`.
- Banner font created once in `Game.__init__` (size 72) to avoid changing `draw()`'s signature.

Code Changes

1. Module-level `_banner` dict (added below `_current_wave`):

```
# Banner state: set by on_wave_start(), ticked in update(), drawn in draw().
_banner = {"text": "", "timer": 0.0}
```

2. `on_wave_start()` implementation (replaced `pass` stub):

```
def on_wave_start(wave):
    """Called at the start of every wave; add banners, speed-ups, or palette swaps here."""
    _banner["text"] = f"WAVE {wave}"
    _banner["timer"] = 1.0
```

3. `Game.__init__` — banner font created once:

```
def __init__(self):
    self.stars = [...]
    self.banner_font = pygame.font.Font(None, 72)  # large font for wave banner
    self.reset()
```

4. `update()` — tick the banner timer down:

```
self.time += dt
if _banner["timer"] > 0:
    _banner["timer"] = max(0.0, _banner["timer"] - dt)
```

5. `draw()` — render banner when active:

```
if _banner["timer"] > 0:
    banner_surf = self.banner_font.render(_banner["text"], True, (255, 220, 50))
    screen.blit(banner_surf, banner_surf.get_rect(center=(WIDTH // 2, HEIGHT // 2)))
```

Why All Requirements Are Met

Requirement	How satisfied
Visible "WAVE N" banner	Large 72px yellow text, centered on screen
~1 second duration	`timer = 1.0`, decremented by `dt` each frame
No `time.sleep()`	Pure `dt`-based countdown in `update()`

Requirement	How satisfied
Appears on wave 1	`reset()` → `spawn_wave(1)` → `on_wave_start(1)` sets banner
Every subsequent wave	`update()` → `spawn_wave(N)` → `on_wave_start(N)` resets it
Timer resets each wave	`timer` unconditionally set to `1.0` in `on_wave_start()`
Tasks 1 & 2 preserved	Zero changes to `bezier()` or `enemy_tint()`
Task 4 not touched	`shield_charges()` still `pass` at this stage

Testing

Game launched with no errors:

```
pygame 2.6.1 (SDL 2.28.4, Python 3.13.6)
Hello from the pygame community. https://www.pygame.org/contribute.html
```

Git Push

Commit hash: `db3579c`

Branch: `main`

Files changed: 1 (`Lab4_Galaga/game.py`)

Insertions: 11 | **Deletions:** 1

Commit message:

```
Lab4_Galaga: Task 3 – implement on_wave_start() wave banner

Display a non-blocking 'WAVE N' banner for ~1 second at the start of
every wave (initial wave and all subsequent waves).

Changes:
1. Added module-level _banner dict:
     _banner = {"text": "", "timer": 0.0}
   Same bridge pattern as _current_wave (Task 2) – lets
   on_wave_start() communicate with update()/draw() without
   changing any function signatures or call sites.

2. Implemented on_wave_start(wave):
     _banner['text'] = f'WAVE {wave}'
     _banner['timer'] = 1.0
   Called automatically from spawn_wave() on every wave transition.

3. Added self.banner_font = pygame.font.Font(None, 72) in
   Game.__init__ – created once, keeps draw()'s signature unchanged.

4. Ticked _banner['timer'] down by dt in update() using
   max(0.0, timer - dt) to avoid going negative.

5. Rendered banner in draw() centered on screen in yellow
   (255, 220, 50) whenever timer > 0.

No time.sleep(), no blocking. Tasks 1 and 2 fully preserved.
```

Push output:

```
To https://github.com/diya-122/SE-Lab-Reports.git
   f5eaeb3..db3579c  main -> main
```

Task 4 – Shield Charges (`shield_charges`)

User Prompt

> Implement Task 4 only: `shield_charges(wave)` in `game.py`.

> Requirements:
>
> 1. Grant exactly 1 shield charge on every third wave: waves 3, 6, 9, 12, etc.
> 2. Return 0 on all other waves.
> 3. Inspect how `shield_charges(wave)` is called and how `self.shield` is consumed in `hit_player()`.
> 4. Ensure the shield charge prevents a hit from costing the player a ship.
> 5. Do not change the Bézier fix, enemy tint, wave banner, or other mechanics.
> 6. Make the smallest necessary change.

Inspection of Call Sites and Consumption Logic

Call sites:

```
# reset() – line 139
self.shield = shield_charges(self.wave) or 0   # wave 1 on game start/reset

# update() – line 199
self.shield = shield_charges(self.wave) or 0   # every wave transition
```

The `or 0` means: if `shield_charges` returns any falsy value (`None`, `0`, `pass`), `self.shield` is set to `0`.

Consumption in `hit_player()` (lines 151–164):

```
def hit_player(self):
    if self.invulnerable > 0:
        return                          # already invulnerable, ignore hit
    if self.shield > 0:                 # shield absorbs the hit
        self.shield -= 1
        self.invulnerable = 1.0
        return                          # ← early return, no ship/life lost
    # only reaches here if shield == 0:
    if self.ships > 1:
        self.ships -= 1
    else:
        self.lives -= 1
        if self.lives <= 0:
```

```
            self.state = "lose"
    self.invulnerable = 2.0
```

The shield already works — `self.shield > 0` absorbs the hit, grants 1 second of invulnerability, and returns early. **No changes needed to `hit_player()` or anywhere else.**

Implementation

Single expression replacing the `pass` stub:

```
def shield_charges(wave):
    """Return how many hits the player's shield can absorb this wave, or None to disable the shield.

    Grants 1 shield charge on every third wave (3, 6, 9, 12, …).
    The charge is consumed by hit_player(): self.shield > 0 absorbs one
    hit and returns early, preventing any ship or life from being lost.
    """
    return 1 if wave % 3 == 0 else 0
```

Diff

```
 def shield_charges(wave):
-    """Return how many hits the player's shield can absorb this wave, or None to disable the shield."
-    pass
+    """Return how many hits the player's shield can absorb this wave, or None to disable the shield.
+
+    Grants 1 shield charge on every third wave (3, 6, 9, 12, …).
+    The charge is consumed by hit_player(): self.shield > 0 absorbs one
+    hit and returns early, preventing any ship or life from being lost.
+    """
+    return 1 if wave % 3 == 0 else 0
```

Logic Test – Waves 1 Through 9

Wave	shield_charges	Expected	
1	0	0	OK
2	0	0	OK
3	1	1	OK
4	0	0	OK
5	0	0	OK
6	1	1	OK
7	0	0	OK
8	0	0	OK
9	1	1	OK

All 9 waves return the correct value. ■

Testing

Game launched with no errors:

```
pygame 2.6.1 (SDL 2.28.4, Python 3.13.6)
Hello from the pygame community. https://www.pygame.org/contribute.html
```

Git Push

Commit hash: `28d3d7b`

Branch: `main`

Files changed: 1 (`Lab4_Galaga/game.py`)

Insertions: 7 | **Deletions:** 2

Commit message:

```
Lab4_Galaga: Task 4 – implement shield_charges(wave)

Grant the player 1 shield charge on every third wave (3, 6, 9, 12, …)
and 0 charges on all other waves.

Change (single expression):
  Before: pass
  After:  return 1 if wave % 3 == 0 else 0

How it integrates (no other code changed):
- reset() and update() both call:
    self.shield = shield_charges(self.wave) or 0
  so self.shield is set to 1 on wave 3/6/9, 0 elsewhere.
- hit_player() already has:
    if self.shield > 0:
        self.shield -= 1
        self.invulnerable = 1.0
        return   # skips ship/life deduction
  so the shield absorbs exactly one hit per charge, granting
  1 second of invulnerability with no ship or life lost.

Verified for waves 1-9:
  wave 1->0, 2->0, 3->1, 4->0, 5->0, 6->1, 7->0, 8->0, 9->1

Tasks 1 (Bezier fix), 2 (enemy_tint), and 3 (wave banner)
are fully preserved.
```

Push output:

```
To https://github.com/diya-122/SE-Lab-Reports.git
   db3579c..28d3d7b  main -> main
```

Final Commit History

All four tasks were committed and pushed to the `Lab4_Galaga/` folder in the repository https://github.com/diya-122/SE-Lab-Reports.

Commit	Task	Description
`a67af73`	Task 1	Fix cubic Bézier P2 exponent (`u*t` → `u*t*t`)
`f5eaeb3`	Task 2	Wave-aware `enemy_tint()` with 4-tier color palette
`db3579c`	Task 3	`on_wave_start()` – non-blocking "WAVE N" banner (~1 sec
`28d3d7b`	Task 4	`shield_charges()` – 1 charge on every third wave

Summary of Design Patterns Used

All four implementations share a deliberate constraint: **the stub function signatures and their call sites could not be changed**. Each solution was tailored around this:

Task	Constraint	Solution
Task 1	Fix only the math	Identified `u*t` vs `u*t²`, added one `* t`
Task 2	`enemy_tint(kind)` has no wave param	Module-level `_current_wave` bridge variable
Task 3	`on_wave_start(wave)` has no `self`	Module-level `_banner` dict; timer ticked in `update()`
Task 4	`shield_charges(wave)` — pure function	Single `wave % 3 == 0` expression; existing `hit_player()` lo

---

Document generated: 15 October 2026

All commit hashes and test outputs are taken directly from the session.